

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе №1

Вариант № 9

Выполнил
студент группы 250501:
Снитко Д.А.

Проверил:
Шеменков В.В.

МИНСК 2025

1 ЦЕЛЬ РАБОТЫ И ОПИСАНИЕ УСТРОЙСТВА

Целью данной лабораторной работы является описание комбинационного устройства на языке VHDL.

Устройство реализует алгоритм, который позволяет перейти от двоично-десятичного представления числа перейти к one-hot представлению числа.

На рисунке 1.1 изображена схема устройства. На рисунке 1.2 приведена таблица истинности для данного устройства.

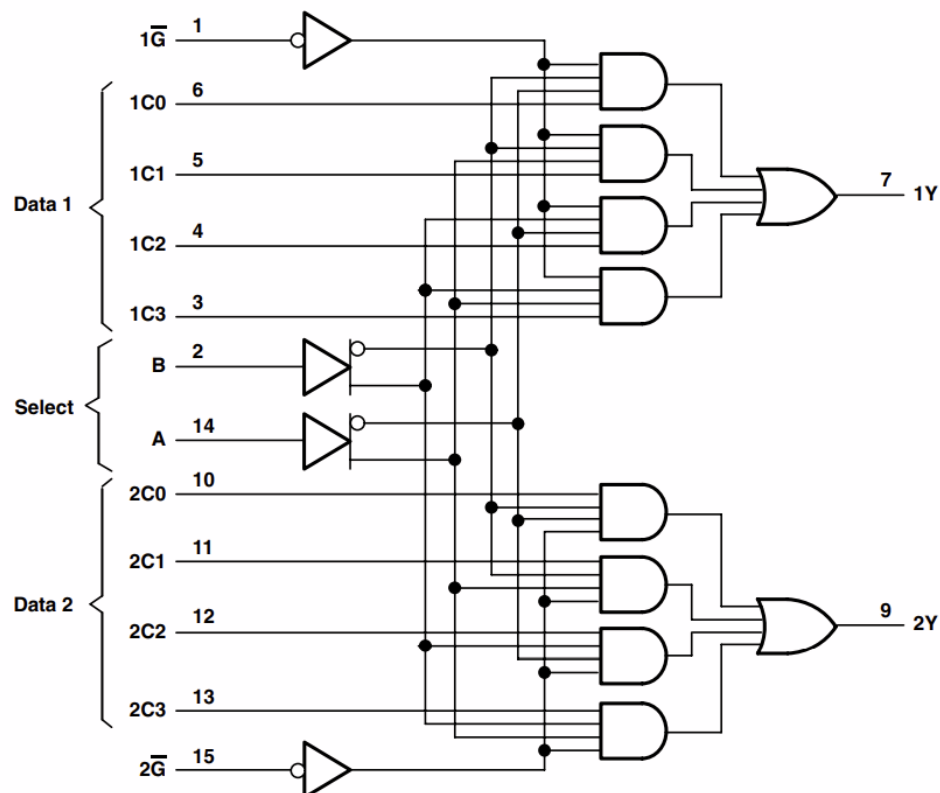


Рисунок 1.1 – Схема устройства

SELECT INPUTS		DATA INPUTS				STROBE	OUTPUTS
B	A	C0	C1	C2	C3	$\bar{G}$	Y
X	X	X	X	X	X	H	L
L	L	L	X	X	X	L	L
L	L	H	X	X	X	L	H
L	H	X	L	X	X	L	L
L	H	X	H	X	X	L	H
H	L	X	X	L	X	L	L
H	L	X	X	H	X	L	H
H	H	X	X	X	L	L	L
H	H	X	X	X	H	L	H

Рисунок 1.2 – таблица истинности устройства

2 ОПИСАНИЕ НА ЯЗЫКЕ VHDL

Используя таблицу истинности можно легко описать устройство на языке описания аппаратуры VHDL.

2.1 Листинг кода параллельного устройства

```
library ieee;
use ieee.std_logic_1164.all;

entity dual_mux is
    port (
        S0, S1 : in std_logic;           -- Селекторы
        для выбора входа
        D1_0, D1_1, D1_2, D1_3 : in std_logic; -- Входы для
        первого мультиплексора (Data1)
        D2_0, D2_1, D2_2, D2_3 : in std_logic; -- Входы для
        второго мультиплексора (Data2)
        Y1, Y2 : out std_logic           -- Выходы
        мультиплексоров
    );
end entity dual_mux;

architecture dataflow of dual_mux is
begin
    -- Вычисление Y1
    Y1 <= (D1_0 and not S0 and not S1) or -- Выбор D1_0 при
    S1S0 = "00"
           (D1_1 and      S0 and not S1) or -- Выбор D1_1 при
    S1S0 = "01"
           (D1_2 and not S0 and      S1) or -- Выбор D1_2 при
    S1S0 = "10"
           (D1_3 and      S0 and      S1);  -- Выбор D1_3 при
    S1S0 = "11"

    -- Вычисление Y2
    Y2 <= (D2_0 and not S0 and not S1) or
```

```

        (D2_1 and      S0 and not S1) or
        (D2_2 and not S0 and      S1) or
        (D2_3 and      S0 and      S1);
end architecture dataflow;

```

2.2 Листинг кода последовательного устройства

```

library ieee;
use ieee.std_logic_1164.all;

entity dual_mux is
    port (
        S0, S1 : in std_logic;
        D1_0, D1_1, D1_2, D1_3 : in std_logic;
        D2_0, D2_1, D2_2, D2_3 : in std_logic;
        Y1, Y2 : out std_logic
    );
end entity dual_mux;

architecture behavioral of dual_mux is
begin
    process (S0, S1, D1_0, D1_1, D1_2, D1_3, D2_0, D2_1, D2_2,
D2_3)
        variable sel : std_logic_vector(1 downto 0); -- Теперь
переменная
        variable temp_y1, temp_y2 : std_logic;
    begin
        sel := S1 & S0; -- Мгновенное присваивание
        case sel is
            when "00" =>
                temp_y1 := D1_0;
                temp_y2 := D2_0;
            when "01" =>

```

```

        temp_y1 := D1_1;
        temp_y2 := D2_1;
    when "10" =>
        temp_y1 := D1_2;
        temp_y2 := D2_2;
    when others =>
        temp_y1 := D1_3;
        temp_y2 := D2_3;
    end case;
    Y1 <= temp_y1;
    Y2 <= temp_y2;
end process;
end architecture behavioral;

```

2.3 Листинг кода теста устройств

```

library ieee;
use ieee.std_logic_1164.all;

entity tbench_dual_mux_bh is
end entity tbench_dual_mux_bh;

architecture sim of tbench_dual_mux_bh is
    signal S0_tb, S1_tb : std_logic := '0';
    signal D1_0_tb, D1_1_tb, D1_2_tb, D1_3_tb : std_logic :=
'0';
    signal D2_0_tb, D2_1_tb, D2_2_tb, D2_3_tb : std_logic :=
'0';
    signal Y1_tb, Y2_tb : std_logic;

    component dual_mux
        port (
            S0, S1 : in std_logic;

```

```

        D1_0, D1_1, D1_2, D1_3 : in std_logic;
        D2_0, D2_1, D2_2, D2_3 : in std_logic;
        Y1, Y2 : out std_logic
    );
end component;

begin

    uut: dual_mux port map (
        S0 => S0_tb, S1 => S1_tb,
        D1_0 => D1_0_tb, D1_1 => D1_1_tb, D1_2 => D1_2_tb, D1_3
=> D1_3_tb,
        D2_0 => D2_0_tb, D2_1 => D2_1_tb, D2_2 => D2_2_tb, D2_3
=> D2_3_tb,
        Y1 => Y1_tb, Y2 => Y2_tb
    );

    stim_proc: process
    begin
        -- Инициализация: Все входы в 0
        S1_tb <= '0'; S0_tb <= '0';
        D1_0_tb <= '0'; D1_1_tb <= '0'; D1_2_tb <= '0'; D1_3_tb
<= '0';
        D2_0_tb <= '0'; D2_1_tb <= '0'; D2_2_tb <= '0'; D2_3_tb
<= '0';
        wait for 100 ns;

        -- Тест 1: S1S0 = "00", выбираем D1_0 и D2_0
        S1_tb <= '0'; S0_tb <= '0';
        D1_0_tb <= '1'; D1_1_tb <= '0'; D1_2_tb <= '0'; D1_3_tb
<= '0';
        D2_0_tb <= '1'; D2_1_tb <= '0'; D2_2_tb <= '0'; D2_3_tb
<= '0';
        wait for 100 ns;

        assert (Y1_tb = '1' and Y2_tb = '1') report "Ошибка: Y1
или Y2 не '1' при S=00" severity error;
    end process;
end;

```

```

-- Тест 2: S1S0 = "01", выбираем D1_1 и D2_1
S1_tb <= '0'; S0_tb <= '1'; -- Исправлено: S1S0 = "01"
D1_0_tb <= '0'; D1_1_tb <= '1'; D1_2_tb <= '0'; D1_3_tb
<= '0';
D2_0_tb <= '0'; D2_1_tb <= '1'; D2_2_tb <= '0'; D2_3_tb
<= '0';

wait for 100 ns;

assert (Y1_tb = '1' and Y2_tb = '1') report "Ошибка: Y1
или Y2 не '1' при S=01" severity error;

-- Тест 3: S1S0 = "10", выбираем D1_2 и D2_2
S1_tb <= '1'; S0_tb <= '0'; -- Исправлено: S1S0 = "10"
D1_0_tb <= '0'; D1_1_tb <= '0'; D1_2_tb <= '1'; D1_3_tb
<= '0';
D2_0_tb <= '0'; D2_1_tb <= '0'; D2_2_tb <= '1'; D2_3_tb
<= '0';

wait for 100 ns;

assert (Y1_tb = '1' and Y2_tb = '1') report "Ошибка: Y1
или Y2 не '1' при S=10" severity error;

-- Тест 4: S1S0 = "11", выбираем D1_3 и D2_3
S1_tb <= '1'; S0_tb <= '1';
D1_0_tb <= '0'; D1_1_tb <= '0'; D1_2_tb <= '0'; D1_3_tb
<= '1';
D2_0_tb <= '0'; D2_1_tb <= '0'; D2_2_tb <= '0'; D2_3_tb
<= '1';

wait for 100 ns;

assert (Y1_tb = '1' and Y2_tb = '1') report "Ошибка: Y1
или Y2 не '1' при S=11" severity error;

-- Инициализация: Все входы в 0
S1_tb <= '0'; S0_tb <= '0';
D1_0_tb <= '0'; D1_1_tb <= '0'; D1_2_tb <= '0'; D1_3_tb
<= '0';

```

```

        D2_0_tb <= '0'; D2_1_tb <= '0'; D2_2_tb <= '0'; D2_3_tb
<= '0';

        wait for 100 ns;

        -- Тест 5: S1S0 = "00", выбираем D1_0 и D2_0
        S1_tb <= '0'; S0_tb <= '0';
        D1_0_tb <= '0'; D1_1_tb <= '0'; D1_2_tb <= '0'; D1_3_tb
<= '1';
        D2_0_tb <= '0'; D2_1_tb <= '0'; D2_2_tb <= '0'; D2_3_tb
<= '1';

        wait for 100 ns;

        assert (Y1_tb = '0' and Y2_tb = '0') report "Ошибка: Y1
или Y2 не '1' при S=00" severity error;

        -- Тест 6: S1S0 = "01", выбираем D1_1 и D2_1
        S1_tb <= '0'; S0_tb <= '1'; -- Исправлено: S1S0 = "01"
        D1_0_tb <= '0'; D1_1_tb <= '0'; D1_2_tb <= '1'; D1_3_tb
<= '0';
        D2_0_tb <= '0'; D2_1_tb <= '0'; D2_2_tb <= '1'; D2_3_tb
<= '0';

        wait for 100 ns;

        assert (Y1_tb = '0' and Y2_tb = '0') report "Ошибка: Y1
или Y2 не '1' при S=01" severity error;

        -- Тест 7: S1S0 = "10", выбираем D1_2 и D2_2
        S1_tb <= '1'; S0_tb <= '0'; -- Исправлено: S1S0 = "10"
        D1_0_tb <= '0'; D1_1_tb <= '1'; D1_2_tb <= '0'; D1_3_tb
<= '0';
        D2_0_tb <= '0'; D2_1_tb <= '1'; D2_2_tb <= '0'; D2_3_tb
<= '0';

        wait for 100 ns;

        assert (Y1_tb = '0' and Y2_tb = '0') report "Ошибка: Y1
или Y2 не '1' при S=10" severity error;

        -- Тест 8: S1S0 = "11", выбираем D1_3 и D2_3
        S1_tb <= '1'; S0_tb <= '1';

```



```

        D1_0_tb <= '1'; D1_1_tb <= '0'; D1_2_tb <= '0'; D1_3_tb
<= '0';

        D2_0_tb <= '1'; D2_1_tb <= '0'; D2_2_tb <= '0'; D2_3_tb
<= '0';

        wait for 100 ns;

        assert (Y1_tb = '0' and Y2_tb = '0') report "Ошибка: Y1
или Y2 не '1' при S=11" severity error;

        -- Сброс всех сигналов
        S1_tb <= '0'; S0_tb <= '0';

        D1_0_tb <= '0'; D1_1_tb <= '0'; D1_2_tb <= '0'; D1_3_tb
<= '0';

        D2_0_tb <= '0'; D2_1_tb <= '0'; D2_2_tb <= '0'; D2_3_tb
<= '0';

        wait for 100 ns;

        assert (Y1_tb = '0' and Y2_tb = '0') report "Ошибка: Y1
или Y2 не '0' после сброса" severity error;

        report "Завершено" severity note;

        wait;

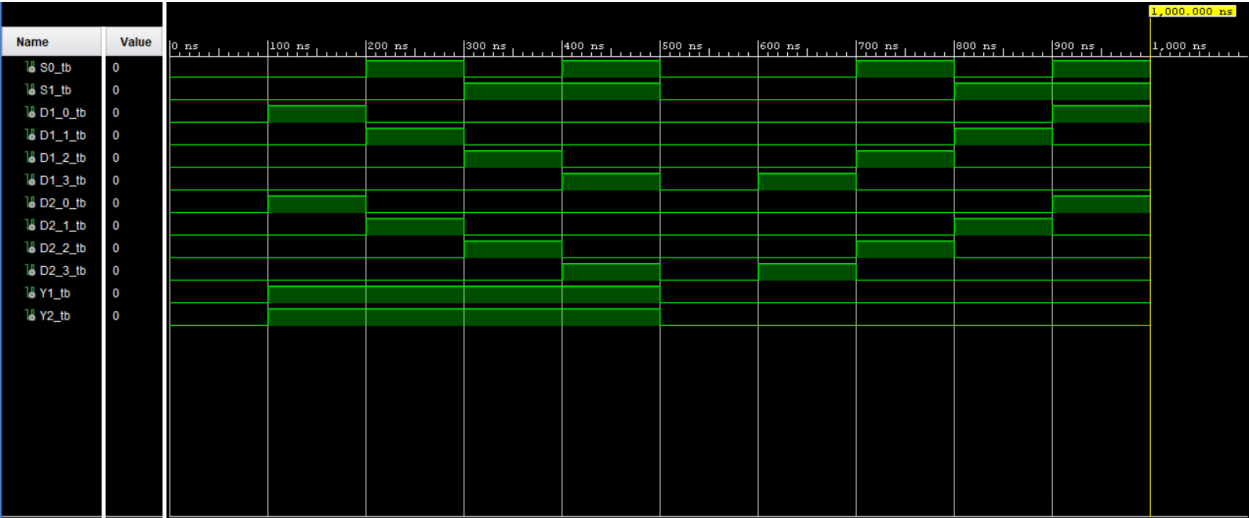
    end process;

end architecture sim;

```

3 ВРЕМЕННАЯ ДИАГРАММА СИМУЛЯЦИИ

Временная диаграмма последовательного оператора:



Временная диаграмма параллельного оператора:

